Agent Harness Documentation 202605

Agent Harness Local Reporting

2026-05-13T00:00:00+01:00

Agent Harness Documentation 202605

Classification: Local workspace documentation. Review before external publication.

Generated: 2026-05-13T00:00:00+01:00

Purpose: Forwardable maintainer handover manual for the Agent Harness local AI control-plane project. Reader journey: understand the project boundary, run one safe session, choose an operator workflow, inspect evidence, understand authority and safety gates, and recover when work is blocked.

Workspace note: This report was generated from the local repository at `/Users/oscarxue/Documents/harness`. At generation time, the worktree contained existing modified and untracked files. This report is therefore a local documentation snapshot, not a clean release tag.

Reader Orientation

Agent Harness is a local-first supervised agent runtime. It provides a command-line and terminal user interface for declarative agents, explicit tasks, durable local state, registered execution adapters, evidence inspection, local memory, runtime controls, and bounded Codex-assisted planning or editing.

The project exists because AI workers need a local control plane around them. Provider tools and agent subprocesses can be useful, but they should not silently decide which files they may touch, when hosted data boundaries are acceptable, whether a diff should apply back to the active repository, or whether evidence is good enough for promotion. Agent Harness keeps those decisions visible through project state, approvals, leases, adapter metadata, artifacts, tests, and operator-facing commands.

Read this report in sequence the first time. It follows the same handover style as the Toloclaw report: tutorial first, then explanation, how-to guidance, reference tables, and appendices.

The practical model is simple: the operator keeps final authority; Harness owns local state, policy checks, approvals, runtime controls, evidence, and apply-back gates; registered adapters perform bounded work; Codex is treated as a supervised external agent backend rather than a raw model provider; active repository mutation requires separate inspected-diff approval.

Tutorial - First Safe Walkthrough

This walkthrough inspects the project without changing behavior.

1. Inspect the worktree before trusting memory:

   ```
   git status --short
   ```

   Preserve unrelated dirty files. Do not stage or revert files unless they belong to the current task.

2. Inspect the installed command surface:

   ```
   harness --help
   harness --project . --output json
   harness home --project . --output json
   ```

These orientation commands are read-only surfaces. They should not create tasks, leases, runs, artifacts, or provider calls.

3. Read the operator documentation:

```
less README.md
less docs/operator_guide.md
less docs/command_catalog.md
less SECURITY.md
```

4. Prefer read-only inspection before dispatch:

```
harness capabilities list --project . --output json
harness daemon adapters --project . --output json
harness tasks list --project .
harness runs --project .
```

5. Only dispatch work through explicit records and leases. The normal pattern is task creation, lease acquisition, lease inspection, registered adapter dispatch, artifact inspection, and then a separate apply-back decision if needed.

6. Close the loop by running the narrowest meaningful test command for the changed area and checking `git status --short` again.

This first walkthrough should leave the workspace unchanged except for deliberate documentation generation or explicitly requested edits.

Explanation - Authority Model

Agent Harness separates control-plane authority from execution.

The operator is the final authority for hosted data-boundary approval, active repository mutation, apply-back, commits, pushes, broad filesystem access, Docker use, and any future external integration.

Harness is the supervisor and evidence layer. It owns project state under `.harness/`, task and objective records, daemon leases, registered adapter dispatch, approvals, policy checks, runtime controls, adapter breakers, artifacts, traces, progress records, memory records, and apply-back validation.

Adapters are bounded executors. They do not become permission grants merely by existing in metadata. A task without a registered adapter cannot execute. An unknown adapter fails closed. Adapter descriptors are documentation and validation metadata, not authority.

Codex is a supervised external agent backend. Harness does not treat Codex as a raw model provider and does not assume Codex internal actions are Harness-native tool calls. Supervision occurs through isolated workspaces, subprocess flags where available, captured output, artifacts, git status, diff inspection, policy validation, and explicit apply-back approval.

Memory is local operator context only. Memory can inform a visible prompt or report, but it cannot grant tools, satisfy approvals, permit hosted execution, authorize Docker, weaken policy, or approve active repository changes.

Reference - Project Inventory

Item	Current value
Package name	`agent-harness`
Version	`1.8.0`
CLI entrypoint	`harness`
Description	Local-first agent harness with durable queue and control-plane evidence
Primary dependencies	`pydantic`, `pyyaml`, `textual`, `typer`
Source/spec files counted under `src/harness`	91
Built-in YAML specs	33
Top-level test files	46

Item	Current value
Main operator docs	`README.md`, `docs/operator_guide.md`, `docs/command_catalog.md`, `docs/smoke_checklist.md`, `SECURITY.md`

Reference - Operator Surfaces

Surface	Purpose	Authority
`harness`	Unified Textual operator app with dashboard and prompt	Routes through explicit Harness actions
`harness --plain`	Line-oriented fallback for terminals and tests	Same control-plane model
`harness --output json`	Read-only app context probe	No execution
`harness home`	Project state snapshot	Read-only
`harness capabilities`	Inspect registered adapter capability metadata	Read-only
`harness memory`	Save, inspect, and forget explicit local notes	Local context only
`harness progress`	Inspect objective/task/lease/run progress	Read-only
`harness daemon run-once`	Select and lease ready work	Lease only
`harness daemon execute`	Dispatch an already leased task to a registered adapter	Bounded by adapter, policy, approval, and controls
`harness objectives run`	Bounded runner over an existing task graph	Does not create new tasks or bypass approvals

Reference - Registered Workflow Modes

Harness uses explicit task types, adapter metadata, and operator templates rather than hidden free-form automation.

Workflow	Adapter / task shape	What it does	What it does not do
Read-only repo summary	`read_only_summary` / `read_only_repo_summary`	Produces supervised repository summary evidence	Does not mutate the repo or use local fallback
Repo planning	`repo_planning` / `repo_planning`	Uses Codex read-only sandbox mode for planning evidence	Fails if read-only policy detects active repo changes
Codex isolated edit	`codex_isolated_edit` / `codex_code_edit`	Runs Codex in an isolated workspace and captures diff evidence	Does not apply back without separate inspected approval
Dry run	`dry_run` / `phase_1a_test`	Produces deterministic adapter evidence for local tests	Does not call providers or mutate source
Docker tests	Direct sandboxed test execution	Runs tests in a sanitized workspace where implemented	Does not mount the active repo directly or enable network by default
Objective autonomy	Existing objective graph	Leases and dispatches ready tasks within budget	Does not ask a model to expand the graph or create tasks

Reference - Built-In Specs

Harness ships declarative built-in specs for coding, quant, and personal workbenches.

Category	Examples
Workbenches	`coding`, `quant`, `personal`
Coding agents	`coding_orchestrator`, `repo_inspector`, `code_editor`, `test_runner`, `implementation_reviewer`, `security_reviewer`, `factuality_reviewer`
Quant agents	`quant_orchestrator`, `quant_researcher`, `commodities_researcher`, `equities_researcher`, `backtest_engineer`, `risk_reviewer`, `leakage_reviewer`, `statistical_validity_reviewer`
Personal agents	`personal_orchestrator`, `job_researcher`
Policy specs	`model_profiles.yaml`, `tool_policies.yaml`, `memory_scopes.yaml`

Imported project agents remain declarative metadata. Importing an agent does not grant new tools, create tasks, create runs, or start background work.

How To - Run A Safe Operator Session

Start with the unified app:

```
harness --project .
```

For a terminal-safe fallback:

```
harness --project . --plain
```

For a foreground action flow suitable for testing:

```
harness --project . --plain --codex-like
```

Inside the app, natural-language requests such as "summarize this repo", "plan how to improve the CLI", and "fix the failing test with Codex" are interpreted into explicit Harness actions. Before work is created or dispatched, the app should show the interpreted intent, proposed action, equivalent commands, safety boundary, required approvals, and confirmation prompt.

The invariant is: user asks, Harness proposes, Harness records or leases or dispatches through registered adapters, evidence returns to the operator.

How To - Plan Or Edit With Codex

Codex planning and editing require hosted data-boundary approval because repository context may leave the machine.

Create a scoped approval profile before a supervised Codex edit:

```
harness approvals add --backend codex_cli --data-boundary hosted_provider --project . --task-types codex_co
```

Run an isolated edit:

```
harness run "Modify only scratch_codex_edit.py. Add a docstring inside greet(). Do not create, delete, or m
```

The active project remains unchanged until apply-back is approved. Apply-back is based on the inspected, sanitized, validated diff, not on Codex final messages, stdout, stderr, or event text.

How To - Inspect Evidence

Use these commands after dispatch:

```
harness runs --project .
harness show <run_id> --project . --output json
harness artifacts list <run_id> --project .
harness artifacts inspect <artifact_id> --project .
harness progress --objective <objective_id> --project . --output json
harness policy explain --subject-kind task --subject-id <task_id> --project . --output json
```

Evidence should answer: what was requested, which task or objective owned it, which adapter executed it, which approvals applied, which sandbox or isolation profile was used, which artifacts were produced, what tests or checks ran, and what remains blocked or risky.

How To - Close Work

Before closing a task:

1. Inspect the diff and confirm that changed files belong to the task.
2. Run focused tests for the touched behavior.
3. Run security or integrity checks when authority, policy, approvals, adapters, memory, Docker, or apply-back behavior changed.
4. Confirm that generated artifacts are intentional.
5. Preserve unrelated dirty files.
6. Commit only related files when a commit is requested.

Suggested focused commands:

```
python3 -m pytest tests/test_cli_smoke.py -q
python3 -m pytest tests/test_approvals.py tests/test_codex_apply_back_c3.py -q
python3 -m pytest tests/test_security_regression_matrix.py -q
harness security check --project . --output json
harness evals run --suite security-layer --project . --output json
```

Explanation - Safety Model

Harness uses a four-plane local security model.

Plane	Role
Policy and approvals	Decide whether a task, adapter, backend, hosted boundary, Docker path, or apply-back path may proceed
Runtime controls and breakers	Locally narrow execution by disabling risky categories or pausing repeatedly failing adapters
Sandbox, profile, and evidence boundaries	Describe where execution may run and what proof is recorded
Context, provenance, integrity, and detection	Make untrusted inputs and generated evidence inspectable without granting permissions

Important boundaries:

- Do not use `OPENAI_API_KEY` or add paid API fallback.
- Do not add hidden hosted fallback.
- Do not expose secrets.
- Do not modify `.harness/`, `.git/`, `.env*`, `*.pem`, `*.key`, `*.sqlite`, or `secrets/` through normal tool or apply-back paths.
- Treat Codex as a supervised external backend.
- Keep Docker network disabled by default.
- Keep active repository apply-back denied unless the inspected-diff approval path approves it.

Reference - Storage Surfaces

Path	Meaning	Handling rule
`.harness/`	Private local runtime state, approvals, queues, runs, memory, leases	Preserve; do not publish without review
`src/harness/`	Python package implementation	Change with focused tests

Path	Meaning	Handling rule
`src/harness/builtin_specs/`	Built-in declarative agents, workbenches, profiles, policies	Validate with spec tests
`docs/`	Operator guides, command catalog, plans, generated documentation	Keep docs aligned with behavior
`tests/`	Unit and behavior tests	Use focused tests for touched modules
`assets/tui/`	Static local TUI art source	Regenerate through explicit command only

Reference - Command Families

Family	Purpose	When to use
Orientation	`harness`, `home`, `doctor`, `quickstart`	Start or diagnose a local session
Agents and specs	`agents`, `specs`	Scaffold, validate, import, inspect, and preview declarative agents
Objectives and tasks	`objectives`, `tasks`	Create and inspect local work records
Daemon control	`daemon`	Lease, inspect, dispatch, recover, and stop controlled work
Capabilities	`capabilities`	Inspect available adapter-backed actions
Approvals	`approvals`	Create scoped hosted-boundary approvals
Runtime controls	`controls`	Disable or re-enable local adapter categories and reset breakers
Evidence	`runs`, `show`, `artifacts`, `progress`, `policy explain`	Inspect what happened and why
Security and evals	`security`, `evals`, `integrity`	Audit local evidence and safety invariants
Memory	`memory`	Store and inspect explicit local operator notes

Recovery Runbook

If a command is blocked, inspect the stable reason rather than retrying with broader authority:

```
harness daemon inspect-lease <lease_id> --project . --output json
harness capabilities inspect <adapter_id> --project . --output json
harness policy explain --subject-kind task --subject-id <task_id> --project . --output json
```

Common blocked states include missing approval, disabled adapter, unsafe metadata, unknown adapter, sandbox profile mismatch, breaker open, and forbidden path or secret-like content.

If Codex editing fails, check hosted-boundary approval, Codex CLI capability, dirty repo refusal, isolated diff policy, and apply-back approval state.

If Docker tests fail, inspect whether the sanitized workspace, disabled network, mounted outputs, and command evidence match the expected sandbox profile.

If memory or context looks wrong, remember that memory is non-authoritative. Correct the local note, but do not treat memory as policy evidence.

Reference Appendices

Documentation Inventory

- `README.md`

- SECURITY.md
- docs/operator_guide.md
- docs/command_catalog.md
- docs/smoke_checklist.md
- docs/plans/autonomous_chat_runtime_full_implementation_plan.md
- docs/plans/autonomy_pr1_implementation_plan.md
- docs/plans/opencode_style_llm_chat_plan.md

Test Surface

Agent Harness top-level test files detected: 46.

- tests/test_action_proposals.py
- tests/test_agent_authoring_v0_7.py
- tests/test_approvals.py
- tests/test_autonomy.py
- tests/test_capabilities_v1_8.py
- tests/test_chat_model.py
- tests/test_chat_tools.py
- tests/test_cli_smoke.py
- tests/test_codex_apply_back_c3.py
- tests/test_codex_backend.py
- tests/test_codex_code_edit_c2.py
- tests/test_codex_runner_phase_1c.py
- tests/test_config.py
- tests/test_context_pack.py
- tests/test_docker_image_manager_phase_4.py
- tests/test_docker_sandbox_phase_3a.py
- tests/test_docker_tests_cli_phase_3a.py
- tests/test_docs_phase_3d.py
- tests/test_edit_runner_phase_2a.py
- tests/test_effective_policy_v0_3_5.py
- tests/test_evals_traces_v0_3_5.py
- tests/test_golden_evidence_v0_1.py
- tests/test_isolation_c1.py
- tests/test_local_backend.py
- tests/test_memory_v1_8.py
- tests/test_objective_runner.py
- tests/test_operator_chat_path.py
- tests/test_packaging_v1_2.py
- tests/test_patch_tool.py
- tests/test_paths_security.py
- tests/test_progress_v1_8.py
- tests/test_protocol.py
- tests/test_readonly_tools.py
- tests/test_registry_v0_2.py
- tests/test_reviewer_workflows.py
- tests/test_runner_phase_1b.py
- tests/test_sandbox_profiles.py
- tests/test_security_regression_matrix.py
- tests/test_spec_diff_v0_2.py
- tests/test_spec_effective_preview_v0_2.py
- tests/test_spec_export_v0_2.py
- tests/test_spec_loader_v0_2.py
- tests/test_specs_v0_2.py
- tests/test_sqlite_store.py
- tests/test_tool_capabilities_v0_3_5.py
- tests/test_tui_pixel_art.py

Standards Used For This Report

Source	Role in this report
Diataxis	Documentation structure by tutorial, how-to, reference, and explanation
Google Developer Style Guide	Plain language, scannability, accessibility, and active voice
Local Harness docs	Project behavior, command surface, security boundaries, and operator workflows

Forwarding Note

This PDF is suitable as a project orientation document after review. Before forwarding outside the local project group, check whether any local paths, internal plans, or worktree-state notes should be removed.